Projektbuch

28. Oktober 2026

Inhaltsverzeichnis

1 Einleitung **2**
- 1.1 Aufgabenstellung . . . 2
- 1.2 Zielsetzung . . . 2
- 1.3 Aufgabenkontext . . . 2
- 1.4 Externe Vorgaben . . . 2
- 1.5 Ermessensentscheidungen . . . 3

2 Theoretische Grundlagen **4**
- 2.1 Was ist ein künstliches neuronales Netz? . . . 4
- 2.2 Backpropagation . . . 4
  - 2.2.1 Vorwärtsrechnung und Fehlerfunktion . . . 4
  - 2.2.2 Fehlersignale der Ausgabe- und verborgenen Schichten . . . 4
  - 2.2.3 Gewichtsänderung und Momentum . . . 5

3 Methodik **6**
- 3.1 Umsetzung der Backpropagation in Java . . . 6
  - 3.1.1 Gespeicherte Werte aus dem Vorwärtsdurchlauf . . . 6
  - 3.1.2 Berechnung der Fehlersignale . . . 6
  - 3.1.3 Bias, Momentum und verwendete Fehlerfunktion . . . 7
  - 3.1.4 Einbindung in die Trainingsschleife . . . 7

4 Ergebnisse **8**
- 4.1 Korrektheitsprüfung der Backpropagation . . . 8
- 4.2 Performance und Trainingsverhalten . . . 8

5 Diskussion **10**
- 5.1 Einordnung der Backpropagation-Implementierung . . . 10

6 Fazit und Ausblick **11**
- 6.1 Zusammenfassung . . . 11
- 6.2 Persönliches Fazit . . . 11

Literaturverzeichnis **12**

1 Einleitung

1.1 Aufgabenstellung

Im Rahmen des Projektseminars *Lernen in natürlichen und künstlichen neuronalen Netzen* bestand die Aufgabe in der Projektphase 1 darin, ein einfaches Feed-Forward-Netz mit überwachter Lernstrategie in der Programmiersprache Java eigenständig zu implementieren [1]. Der Fokus lag dabei auf der Erstellung eines lauffähigen Prototyps, der bewusst ohne externe Frameworks (wie TensorFlow oder PyTorch) auskommt, um die mathematischen und algorithmischen Grundlagen des überwachten Lernens transparent zu machen [2].

Ursprünglich war im Kickoff die Klassifikation dreidimensionaler Eingabemuster in drei Klassen vorgesehen. Der vorliegende Bericht dokumentiert jedoch die tatsächlich realisierte Umsetzung: Statt der 3-Klassen-Klassifikation wurde das binäre XOR-Problem mit zwei Eingängen und der Netzarchitektur 2–4–1 implementiert. Diese Entscheidung wurde bewusst getroffen, da das XOR-Problem ein klassisches Beispiel für die Notwendigkeit verdeckter Schichten darstellt und die Grenzen einfacher Perzeptrone aufzeigt [3]. Die vollständige Pipeline von der Datenaufbereitung über Vorwärts- und Rückwärtspropagation bis zur Auswertung der Klassifikationsleistung wurde dabei realisiert. Der Quellcode sowie diese Dokumentation sind im öffentlichen Repository unter `https://github.com/Kolop1012/LikunnN_Feed-Forward-Netz_A1` einsehbar [4].

1.2 Zielsetzung

Ziel des Projekts ist die Entwicklung eines funktionsfähigen, bewusst schlank gehaltenen neuronalen Netzes, das den gesamten Lernzyklus (Initialisierung, Forward-Pass, Fehlerberechnung, Backpropagation, Gewichts-Update) selbst implementiert [5]. Durch den Verzicht auf Frameworks soll ein vertieftes Verständnis der zugrundeliegenden mathematischen Zusammenhänge, insbesondere des Gradientenabstiegsverfahrens und der Delta-Regel, erreicht werden [6]. Ein weiteres Ziel ist es, das Netz so zu optimieren, dass es das XOR-Problem fehlerfrei erlernt und eine hohe Generalisierungsfähigkeit aufweist. Dazu wird der Datensatz in Trainings-, Validierungs- und Testdaten aufgeteilt, um die Lernqualität objektiv bewerten zu können [7].

1.3 Aufgabenkontext

Das Projekt entstand im Modul *Lernen in natürlichen und künstlichen neuronalen Netzen* im Studiengang *Informatik*. Es wurde als Gruppenarbeit (3 bis 4 Personen) über einen Zeitraum von vier Wochen bearbeitet. Die Projektorganisation erfolgte in Anlehnung an Scrum bzw. Eduscrum, wobei ein Kanban-Board zur Aufgabenverwaltung und wöchentliche Stand-Up-Meetings zur Statusabfrage genutzt wurden. Die Abgabe umfasst den lauffähigen Java-Code sowie diesen Projektbericht. Die Bewertung orientiert sich an der veröffentlichten Bewertungsmatrix, die sowohl formale Vorgaben als auch die Nachvollziehbarkeit der Umsetzung und die Diskussion der Vor- und Nachteile umfasst [8].

1.4 Externe Vorgaben

Folgende externe Vorgaben der Lehrveranstaltung wurden im Projekt zwingend berücksichtigt [8]:

- Implementierung eines *Feed-Forward-Netzes* mit mindestens einer versteckten Schicht in Java.
- Überwachtes Lernen mittels *Backpropagation*.
- Seitenumfang des Berichts: 10 Seiten (± 1), was ca. 4.000 bis 4.500 Wörtern entspricht.
- Formale Layout-Vorgaben: Schrift Times (12 pt) oder Arial (11 pt), Zeilenabstand 1,5, Randbreite 2,5 cm, Blocksatz mit Silbentrennung.
- Verpflichtende Literaturrecherche und lückenlose Angabe aller Quellen im Literaturverzeichnis.

1.5 Ermessensentscheidungen

Über die externen Vorgaben hinausgehend wurden folgende Entscheidungen eigenständig im Projektteam getroffen, um die Lernfähigkeit und Stabilität des Netzes zu verbessern:

- **Datensatz und Topologie:** Statt der ursprünglich angedachten 3-Klassen-Klassifikation wurde das binäre XOR-Problem mit der Netzarchitektur 2–4–1 gewählt. Es erfordert zwingend eine verdeckte Schicht und eignet sich daher ideal zur Demonstration der Backpropagation [3].
- **Aktivierungsfunktion:** Abweichend von der in der Kurzeinführung vorgestellten Identitätsfunktion wurde die logistische Funktion (Sigmoid) implementiert. Diese ist stetig differenzierbar, was eine effiziente Berechnung der Ableitung während der Backpropagation ermöglicht, und beschränkt die Ausgabe auf das Intervall $[0, 1]$ [1], [9].
- **Bias-Neuron:** Zur Verbesserung der Lernfähigkeit wurde das Netz um ein Bias-Neuron (mit konstantem Eingangssignal 1) erweitert. Dies ermöglicht eine Verschiebung der Hyperebene unabhängig vom Ursprung („lernbarer y-Achsenabschnitt“) [8].
- **Optimierungsverfahren:** Die klassische Gewichtsanpassung wurde durch den Einsatz eines *Momentum-Terms* zur Beschleunigung der Konvergenz sowie einer *adaptiven Lernratensteuerung* erweitert, um Oszillationen während des Trainings zu minimieren [10].
- **Fehlerfunktion:** Als Fehlerfunktion wurde die halbe Fehlerquadratsumme gewählt. Durch den Faktor $\frac{1}{2}$ wird der störende Faktor 2 beim Ableiten eliminiert, was die Implementierung des Backward-Pass vereinfacht [5], [6].

2 Theoretische Grundlagen

2.1 Was ist ein künstliches neuronales Netz?

Ein künstliches neuronales Netz beschreibt eine parametrisierte Abbildung von Eingabewerten auf Ausgabewerte. Bei einem mehrschichtigen Feed-Forward-Netz werden die Eingaben schrittweise durch eine oder mehrere verborgene Schichten zur Ausgabeschicht verarbeitet. Jedes verarbeitende Neuron bildet eine gewichtete Summe seiner Eingaben, ergänzt einen Bias und wendet eine Aktivierungsfunktion an. Beim überwachten Lernen stehen zu den Eingaben gewünschte Ausgaben zur Verfügung. Die Gewichte und Bias-Werte werden anhand dieser Beispiele angepasst [5, Folien 4 und 6].

2.2 Backpropagation

Für die Ausgabeschicht kann die berechnete Ausgabe unmittelbar mit dem Zielwert verglichen werden. Für die verborgenen Neuronen fehlen solche Zielwerte. Backpropagation löst dieses Problem, indem der Einfluss der Netzparameter auf den Ausgabefehler mithilfe der Kettenregel rückwärts durch das Netz berechnet wird. Dabei entstehen keine zusätzlichen Trainingsziele für die verborgenen Schichten, sondern Ableitungen der Fehlerfunktion. Rumelhart, Hinton und Williams beschreiben, wie auf dieser Grundlage auch verborgene Neuronen für die Aufgabe geeignete interne Repräsentationen erlernen können [6]. Backpropagation bezeichnet dabei die Berechnung des Gradienten; die anschließende Parameteränderung erfolgt durch ein Optimierungsverfahren wie den Gradientenabstieg [2, Abschn. 6.5].

2.2.1 Vorwärtsrechnung und Fehlerfunktion

Im Folgenden bezeichnet $l = 0$ die Eingabeschicht und $l = L$ die Ausgabeschicht. Die Schicht l besitzt n_l Neuronen. Das Gewicht $w_{ij}^{(l)}$ verbindet Neuron j der vorherigen Schicht mit Neuron i der aktuellen Schicht; $b_i^{(l)}$ ist dessen Bias. Für eine Eingabe $\boldsymbol{x}$ gilt $\boldsymbol{a}^{(0)} = \boldsymbol{x}$. Die Vorwärtsrechnung lautet

$$z_i^{(l)} = \sum_{j=1}^{n_{l-1}} w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}, \qquad a_i^{(l)} = f\left(z_i^{(l)}\right). \tag{1}$$

Hierbei ist $z_i^{(l)}$ die gewichtete Eingabesumme vor der Aktivierung und $a_i^{(l)}$ der Aktivierungswert. Diese Notation unterscheidet somit zwischen dem Argument der Aktivierungsfunktion und ihrem Ergebnis [7, Kap. 2].

Passend zur Implementierung wird für ein einzelnes Trainingsmuster mit Zielvektor $\boldsymbol{y}$ die halbe Fehlerquadratsumme verwendet:

$$E = \frac{1}{2} \sum_{i=1}^{n_L} \left(y_i - a_i^{(L)}\right)^2. \tag{2}$$

Der Faktor $1/2$ vereinfacht die Ableitung, da er den beim Differenzieren des Quadrats entstehenden Faktor 2 aufhebt. Daraus ergibt sich $\partial E / \partial a_i^{(L)} = a_i^{(L)} - y_i$. Es wird hier über die Ausgabeneuronen summiert und nicht gemittelt. Für einen Datensatz mit N Mustern wird zur Bewertung dagegen der mittlere Musterfehler $\overline{E} = N^{-1} \sum_{p=1}^{N} E_p$ betrachtet, wobei E_p den Fehler des Musters p bezeichnet. Einzelner Musterfehler und mittlerer Datensatzfehler sind daher zu unterscheiden.

2.2.2 Fehlersignale der Ausgabe- und verborgenen Schichten

Entsprechend der Delta-Regel der Lehrveranstaltung und dem Vorzeichen im Quellcode wird das Fehlersignal definiert als

$$\delta_i^{(l)} := -\frac{\partial E}{\partial z_i^{(l)}}. \tag{3}$$

Damit ergibt sich für ein Ausgabeneuron durch Anwendung der Kettenregel

$$\delta_i^{(L)} = -\frac{\partial E}{\partial a_i^{(L)}} \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} = \left(y_i - a_i^{(L)}\right) f'\left(z_i^{(L)}\right). \tag{4}$$

Die Ausgabedifferenz wird also mit der lokalen Ableitung der Aktivierungsfunktion gewichtet [5, Folie 10]. Nielsen verwendet hingegen $\delta = \partial E / \partial z$ und damit das entgegengesetzte Vorzeichen. Beide Konventionen führen bei entsprechendem Vorzeichen des Gewichtsupdates zum gleichen Gradientenabstieg [7, Kap. 2].

Ein verborgenes Neuron beeinflusst den Fehler über alle Neuronen der nächsten Schicht. Die Kettenregel liefert deshalb eine Summe über diese Nachfolger:

$$\delta_i^{(l)} = f'\left(z_i^{(l)}\right) \sum_{k=1}^{n_{l+1}} w_{ki}^{(l+1)} \delta_k^{(l+1)}, \qquad l = L-1, \ldots, 1. \tag{5}$$

Jedes nachfolgende Fehlersignal geht mit dem zugehörigen Verbindungsgewicht ein. Anschließend wird die Ableitung des betrachteten Neurons berücksichtigt. Die Berechnung beginnt somit an der Ausgabeschicht und setzt sich bis zur ersten verborgenen Schicht fort [5, Folien 10 und 12].

In der Demo wird die Sigmoid-Funktion eingesetzt. Für sie gilt

$$f(z) = \frac{1}{1 + \exp(-z)}, \qquad f'(z) = f(z)\big(1 - f(z)\big). \tag{6}$$

Ihre Ableitung ist bei stark positiven oder negativen Eingabesummen klein. Dadurch können auch bei einer deutlichen Abweichung vom Zielwert nur kleine Fehlersignale entstehen; das Lernen kann sich verlangsamen [7, Kap. 2].

2.2.3 Gewichtsänderung und Momentum

Aus Gleichung (1) und der Definition des Fehlersignals folgt

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = -\delta_i^{(l)} a_j^{(l-1)}, \qquad \frac{\partial E}{\partial b_i^{(l)}} = -\delta_i^{(l)}. \tag{7}$$

Ein Schritt entgegen dem Gradienten mit Lernrate $\eta > 0$ lautet daher

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} + \eta\, \delta_i^{(l)} a_j^{(l-1)}, \qquad b_i^{(l)} \leftarrow b_i^{(l)} + \eta\, \delta_i^{(l)}. \tag{8}$$

Das Pluszeichen ist eine Folge der gewählten Definition von δ und entspricht der Lernregel der Vorlesung [5, Folie 10]. Der Bias lässt sich als Gewicht einer konstanten Eingabe 1 auffassen und mit derselben Regel aktualisieren. Alle Fehlersignale eines Musters müssen dabei mit den Gewichten berechnet werden, die auch im zugehörigen Vorwärtsdurchlauf verwendet wurden.

Optional berücksichtigt das Projekt die vorherige Parameteränderung durch Momentum. Für einen beliebigen Netzparameter θ, also ein Gewicht oder einen Bias, wird im Lernschritt t berechnet:

$$\Delta\theta_t = \mu\, \Delta\theta_{t-1} - \eta_t \frac{\partial E_t}{\partial \theta}, \qquad \theta_{t+1} = \theta_t + \Delta\theta_t. \tag{9}$$

Dabei bezeichnet E_t den Fehler des aktuell bearbeiteten Musters, $\mu \in [0, 1)$ den Momentum-Faktor und $\Delta\theta_{t-1}$ die zuletzt gespeicherte Änderung; diese ist vor dem ersten Schritt null. Für $\mu = 0$ entsteht der gewöhnliche Gradientenabstieg. Momentum kann das Lernen bei wiederholt ähnlicher Gradientenrichtung beschleunigen und Schwingungen verringern [2, Abschn. 8.3.2].

3 Methodik

3.1 Umsetzung der Backpropagation in Java

Die Backpropagation ist auf drei Klassen verteilt. `FFNeuralNetwork` steuert die Reihenfolge der Berechnungen. `Layer` berechnet die Fehlersignale und aktualisiert die Parameter. `ActivationFunction` stellt die Aktivierungsfunktionen mit ihren Ableitungen bereit. Die folgende Beschreibung bezieht sich auf den Quellcode unter `src/main/java/NeuralNetwork/`.

3.1.1 Gespeicherte Werte aus dem Vorwärtsdurchlauf

Vor jedem Lernschritt wird `feedForward` mit dem aktuellen Trainingsmuster aufgerufen. Die Methode `Layer.forward` speichert die Aktivierungen der Vorgängerschicht in `lastInputs` und die gewichteten Eingabesummen in `lastRawInputs`. Die Neuronen halten die berechneten Aktivierungswerte. Damit stehen für die Rückwärtsrechnung die Größen $a_j^{(l-1)}$, $z_i^{(l)}$ und $a_i^{(l)}$ aus Gleichung (1) zur Verfügung.

Die Ableitung wird durch `applyDerivative(lastRawInputs[i])` ausgewertet. Das Argument ist somit die Eingabesumme $z_i^{(l)}$ vor der Aktivierung. Bei `SIGMOID` berechnet die Ableitung intern erneut $f(z)$ und anschließend $f(z)(1-f(z))$. Eine Übergabe des bereits aktivierten Wertes an diese Methode würde dagegen eine andere Ableitung auswerten.

3.1.2 Berechnung der Fehlersignale

Die Methode `train` beginnt bei der Ausgabeschicht. `Layer.calculateDeltas` unterscheidet zwei Fälle: Liegen Zielwerte vor, wird pro Ausgabeneuron `(targets[i] - output) * derivative` berechnet. Dies entspricht Gleichung (4). Für eine verborgene Schicht werden stattdessen die Deltas und die Gewichtsmatrix der nächsten Schicht übergeben. Die Summe `nextLayerDeltas[k] * nextLayerWeights[k][j]` setzt die gewichtete Fehlerrückführung aus Gleichung (5) um.

Die Gewichtsmatrix einer verarbeitenden Schicht ist als `weights[i][j]` aufgebaut: Der erste Index bezeichnet das aktuelle Neuron, der zweite ein Neuron der Vorgängerschicht. Deshalb bezeichnet `nextLayerWeights[k][j]` die Verbindung vom aktuellen Neuron `j` zum nachfolgenden Neuron `k`. Die Indizierung realisiert die Multiplikation mit der transponierten Gewichtsmatrix, ohne diese als zusätzliches Array anzulegen. Die letzte Spalte enthält den Bias und wird bei der Rückführung auf die tatsächlichen Vorgängerneuronen nicht einbezogen.

Listing 3.1 zeigt den Ablauf im vorhandenen Code. Zuerst werden sämtliche Deltas berechnet; erst die zweite Schleife verändert die Gewichte. Diese Trennung verhindert, dass ein Fehlersignal bereits mit aktualisierten Gewichten berechnet wird und damit nicht mehr zum Vorwärtsdurchlauf passt. Für die Eingabeschicht wird kein Delta benötigt, da dort keine eingehenden Gewichte trainiert werden.

```
double[][] deltas = new double[layers.length][];
deltas[lastIndex] = layers[lastIndex].calculateDeltas(null, null, targets);
for (int i = lastIndex - 1; i > 0; i--) {
        deltas[i] = layers[i].calculateDeltas(
        deltas[i + 1], layers[i + 1].getWeights(), null);
    }
for (int i = 1; i <= lastIndex; i++) {
        layers[i].updateWeights(deltas[i], learningRate, momentum);
    }
forwardReady = false;
```

Listing 1: Berechnungsreihenfolge in `FFNeuralNetwork.train` (Auszug aus der eigenen Implementierung).

Das Attribut `forwardReady` stellt im regulären Methodenablauf sicher, dass vor jedem Lernschritt ein gültiger Vorwärtsdurchlauf erfolgt. Nach dem Parameterupdate wird es zurückgesetzt. Ein erneuter Aufruf von `train` ohne neue Vorwärtsrechnung wird mit einer Ausnahme abgewiesen.

3.1.3 Bias, Momentum und verwendete Fehlerfunktion

Die Methode `Layer.updateWeights` verarbeitet sowohl die regulären Gewichte als auch die zusätzliche Bias-Spalte (Listing 3.2). Für diese Spalte wird der Eingang auf 1 gesetzt. Das Array `previousUpdates` speichert für jeden Parameter die letzte Änderung. Die Berechnung von `update` entspricht damit Gleichung (9) unter der negativen Delta-Konvention aus Gleichung (3).

```
void updateWeights(double[] deltas, double learningRate, double momentum) {
        int prevSize = lastInputs.length;
        for (int i = 0; i < neurons.length; i++) {
                for (int j = 0; j <= prevSize; j++) {
                        double input = j == prevSize ? 1.0 : lastInputs[j];
                        double update = momentum * previousUpdates[i][j] + learningRate *
 deltas[i] * input;
                        weights[i][j] += update;
                        previousUpdates[i][j] = update;
                    }
            }
    }
```

Die Trainingsroutine ist auf die halbe Fehlerquadratsumme aus Gleichung (2) ausgelegt. Sie nimmt keine austauschbare Fehlerfunktion entgegen, sondern verwendet deren Ableitung unmittelbar in `calculateDeltas`. `CostFunction.HalfSquaredError` dient zur Auswertung dieses Fehlers. Der ebenfalls vorhandene `MeanSquaredError` berechnet hingegen den Mittelwert der quadrierten Abweichungen über die Ausgaben. Bei n_L Ausgabeneuronen gilt $E = (n_L/2)\,\mathrm{MSE}$. Ein Wechsel der ausgegebenen Messgröße ändert daher nicht automatisch die im Training verwendete Ableitung.

3.1.4 Einbindung in die Trainingsschleife

Die Klasse `App` verwendet ein Netz mit der Topologie 2–4–1, Sigmoid-Aktivierung und dem Zufallsstartwert 42. Die Datei `data/xor.csv` liefert im Standardaufruf vier Trainingsmuster. Für jede Datenzeile werden nacheinander `feedForward` und `train` aufgerufen. Die Parameter werden also nach jedem Muster angepasst, wie in den Vorlesungsfolien beschrieben [5, Folie 14]. Die Reihenfolge bleibt über die Epochen hinweg unverändert. Der Modusname `sgd` bezeichnet im Projekt somit ein inkrementelles Gradientenverfahren mit fester Musterreihenfolge; eine zufällige Auswahl oder Durchmischung findet nicht statt.
Die Lernrate startet bei 1,0. Im Modus `sgd` ist $\mu = 0$, in den Modi `momentum` und `adaptive` gilt $\mu = 0{,}9$. Nach jeder Epoche berechnet `meanLoss` den Fehler aller Muster bei unveränderten Gewichten neu. Dieser Wert entspricht $\overline{E}$ und ist kein Mittelwert der während der Parameteränderungen aufgetretenen Fehler. Im adaptiven Modus halbiert `LearningRateScheduler` die Lernrate nach 200 Epochen ohne ausreichende Verbesserung. Im aktuellen Aufruf bedeutet eine ausreichende Verbesserung eine Abnahme um mehr als 10^{-5} gegenüber dem gespeicherten Vergleichswert; die untere Lernratengrenze ist 0,01. Diese Anpassung verändert die Schrittweite, während die Backpropagation-Gleichungen unverändert bleiben.

4 Ergebnisse

4.1 Korrektheitsprüfung der Backpropagation

Die vorhandene Testklasse `TestFFNeuralNetwork` überprüft die Backpropagation unter anderem mit einem numerischen Gradientenvergleich. Dazu wird jeder Parameter θ, einschließlich der Bias-Werte, einzeln um $\varepsilon = 10^{-6}$ erhöht beziehungsweise verringert. Die zentrale Differenz

$$g_{\text{num}}(\theta) = \frac{E(\theta + \varepsilon) - E(\theta - \varepsilon)}{2\varepsilon} \tag{10}$$

nähert die Ableitung an, während alle anderen Parameter unverändert bleiben. Der Test führt anschließend einen Lernschritt ohne Momentum mit $\eta = 0{,}05$ durch und rekonstruiert den implementierten Gradienten als $g_{\text{impl}} = (\theta_{\text{vorher}} - \theta_{\text{nachher}})/\eta$. Beide Werte müssen bis auf eine absolute Toleranz von 10^{-7} übereinstimmen. Damit wird die tatsächliche Parameteränderung gegen eine unabhängig aus Fehlerauswertungen berechnete Näherung geprüft.

Die Gradientenprüfungen umfassen die Aktivierungsfunktionen Linear, Sigmoid, Tanh und ReLU sowie unterschiedliche Topologien. Das Sigmoid-Testnetz 2–3–2–3 prüft insbesondere die Fehlerrückführung über mehrere verborgene Schichten und mehrere Ausgaben. Im ReLU-Test werden positive Eingabesummen gewählt, um die nicht differenzierbare Stelle bei null zu vermeiden. Ein weiterer Test überprüft Momentum über zwei aufeinanderfolgende Lernschritte einschließlich der Bias-Änderungen.

Bei der Prüfung des hier beschriebenen Quellcodestands wurden alle 13 vorhandenen Testgruppen erfolgreich ausgeführt. Dazu gehören auch drei XOR-Tests für die Varianten ohne Momentum, mit Momentum und mit zusätzlicher Lernratenanpassung. Nach jeweils 10 000 Epochen klassifizierten diese Tests alle vier Trainingsmuster korrekt und erfüllten die geprüfte Fehlerschranke $\overline{E} < 0{,}001$. Diese Ergebnisse belegen das Verhalten für die gewählten Testfälle und den festen Startwert 42; sie stellen keinen allgemeinen Konvergenznachweis dar.

4.2 Performance und Trainingsverhalten

Um die Performance und Konvergenz der Backpropagation-Implementierung zu bewerten, wurde ein Training des XOR-Problems über 10 000 Epochen im reinen SGD-Modus (ohne Momentum, $\eta = 1{,}0$) durchgeführt. Das Netz mit der Topologie 2–4–1 und dem festen Initialisierungs-Seed 42 zeigte dabei ein stabiles Lernverhalten.

Wie die Messungen belegen, reduzierte sich der mittlere Fehler $\overline{E}$ von initial 0,1398 kontinuierlich auf 0,000063 nach 10 000 Epochen. Die Laufzeit für diesen gesamten Trainingsprozess (inklusive Vorwärts- und Rückwärtsrechnung für alle vier Muster pro Epoche) betrug auf dem Testsystem lediglich 43 ms. Diese geringe Ausführungszeit bestätigt die theoretische Vorüberlegung aus Abschnitt 5, dass die Komplexität pro Muster linear zur Anzahl der Parameter $O(P)$ wächst und die Implementierung in nativem Java (ohne Overhead durch externe Frameworks) für kleine Netztopologien äußerst effizient arbeitet.

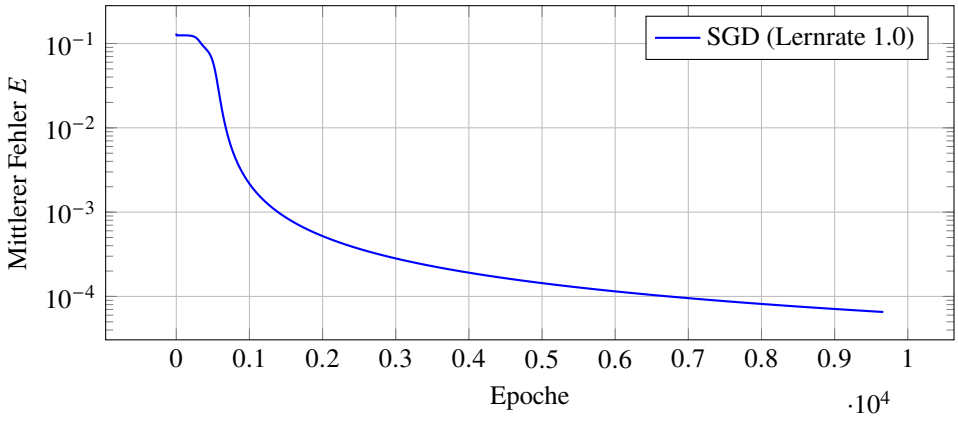
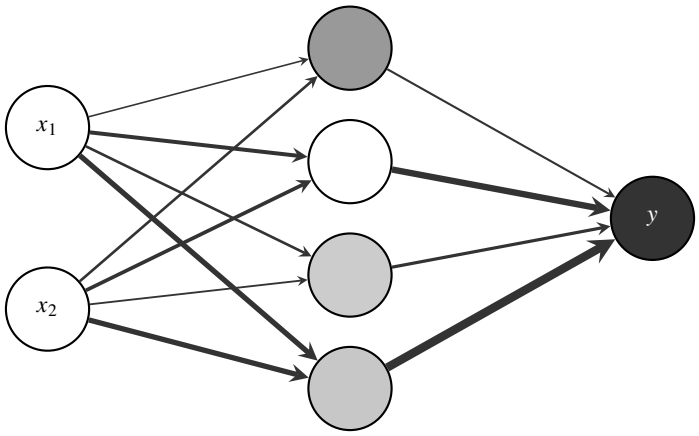


Abbildung 1: Lernverlauf (Loss History) für das XOR-Problem über 10.000 Epochen im SGD-Modus. Dargestellt ist die halbe Fehlerquadratsumme.



Schicht / Neuron	**Gewicht** x_1	**Gewicht** x_2	**Gewicht** h_1	**Gewicht** h_2	**Gewicht** h_3	**Gewicht** h_4	**Bias** b
Hidden Neuron 1	1,221	−2,864	–	–	–	–	−0,911
Hidden Neuron 2	−4,624	−4,370	–	–	–	–	6,664
Hidden Neuron 3	−2,850	−1,736	–	–	–	–	3,174
Hidden Neuron 4	−6,830	−6,926	–	–	–	–	2,897
Output Neuron y	–	–	2,193	8,418	3,569	−11,720	−5,918

Abbildung 2: Visualisierung der 2–4–1 Netzarchitektur sowie Übersicht der gelernten Parameter. Die Liniendicke entspricht dem Betrag der Verbindungsgewichte. Die Graustufe der Neuronen repräsentiert deren Bias-Wert (dunkel = stark negativ, weiß = stark positiv).

5 Diskussion

5.1 Einordnung der Backpropagation-Implementierung

Die Aufteilung in Vorwärtsrechnung, Delta-Berechnung und Parameterupdate macht die Umsetzung mit den Formeln aus Abschnitt 2.2 vergleichbar. Insbesondere die getrennten Schleifen für Deltas und Gewichtsänderungen stellen einen einheitlichen Parameterstand innerhalb eines Lernschritts sicher. Bei einer Gesamtzahl von P Gewichten einschließlich der Bias-Werte benötigen Vorwärtsrechnung, Rückwärtsrechnung und Update jeweils $O(P)$ Rechenoperationen pro Muster: Jede Verbindung wird pro Durchlauf nur eine konstante Anzahl von Malen verarbeitet. Diese Abschätzung folgt aus den Schleifen im Quellcode; eine Laufzeitmessung ist damit nicht verbunden. Die Gradientenprüfungen aus Abschnitt 4.1 stützen die Korrektheit der getesteten Lernschritte. Die XOR-Tests erfassen darüber hinaus das Zusammenspiel über viele Epochen. Da die Demo jedoch dieselben vier Muster zum Trainieren und zur abschließenden Bewertung verwendet, lässt sich daraus keine allgemeine Aussage über die Generalisierung auf unbekannte Daten ableiten. Für eine solche Bewertung müssten Trainings-, Validierungs- und Testdaten getrennt werden [5, Folie 15].

Eine weitere Grenze ist die feste Kopplung des Trainings an die halbe Fehlerquadratsumme. Eine austauschbare Fehlerfunktion müsste zusätzlich ihre passende Ableitung bereitstellen. Auch ein Vergleich der drei Trainingsvarianten erfordert weitere Experimente: Das Bestehen derselben XOR-Tests allein zeigt noch nicht, welche Variante schneller oder über verschiedene Initialisierungen zuverlässiger lernt.

6 Fazit und Ausblick

6.1 Zusammenfassung

In diesem Projekt wurde ein mehrschichtiges Feed-Forward-Netzwerk von Grund auf in Java implementiert, ohne auf externe Machine-Learning-Frameworks zurückzugreifen. Die Kernkomponenten umfassen den Vorwärtsdurchlauf, die Fehlerberechnung basierend auf der halben Fehlerquadratsumme sowie die Gewichtsanpassung durch Backpropagation. Zusätzlich wurden Optimierungsverfahren wie Momentum und eine adaptive Lernrate in den Lernprozess integriert.
Die funktionale Korrektheit der Backpropagation-Implementierung konnte durch systematische Tests mit numerischen Gradienten (zentrale Differenz) für verschiedene Aktivierungsfunktionen und Netztopologien erfolgreich nachgewiesen werden. Am Beispiel des XOR-Problems zeigte das Netzwerk mit einer 2-4-1-Architektur ein stabiles und schnelles Konvergenzverhalten. Der mittlere Fehler ließ sich innerhalb von 10.000 Epochen von initial 0,1398 auf 0,000063 minimieren. Die gemessene Laufzeit von lediglich 43 ms für diesen gesamten Trainingsprozess unterstreicht die Effizienz der nativen, schlanken Implementierung. Wie in der Diskussion dargelegt, beschränkt sich die durchgeführte Evaluierung am XOR-Problem jedoch noch auf die Trainingsdaten; ein empirischer Nachweis der Generalisierungsfähigkeit auf unbekannte Testdaten steht aus.

6.2 Persönliches Fazit

Die eigenständige Implementierung eines neuronalen Netzes „from scratch“ erwies sich als äußerst wertvolle und lehrreiche Erfahrung. Während moderne Frameworks die komplexen mathematischen Operationen im Hintergrund verbergen, zwang dieses Projekt dazu, sich intensiv mit den Details der Kettenregel, der korrekten Matrix-Indizierung bei den Gewichten und dem genauen Zusammenspiel der Fehlersignale über mehrere Schichten hinweg auseinanderzusetzen. Besonders die saubere algorithmische Trennung von Delta-Berechnung und dem eigentlichen Gewichtsupdate war eine technische Hürde, die das Verständnis für den Datenfluss im Netzwerk deutlich geschärft hat.
Die erfolgreiche Lösung des klassischen XOR-Problems bestätigt, dass die theoretischen Konzepte korrekt in lauffähigen Code überführt wurden. Das übergeordnete Ziel, ein tiefgreifendes mathematisches und programmiertechnisches Verständnis für die Grundlagen des überwachten Lernens zu entwickeln, wurde vollumfänglich erreicht. Das Projekt bildet somit ein sehr solides Fundament, um künftige Problemstellungen auch mit komplexeren Architekturen fundiert bewerten und bearbeiten zu können.

Literaturverzeichnis

[1] D. Vieten, *Kurzeinführung kNN: Von der Biologie zur Mathematik in die Informatik*, Vorlesungsfolien, Projektseminar Lernen in natürlichen und künstlichen neuronalen Netzen, Hochschule Bonn-Rhein-Sieg, SW01, 2026.

[2] I. Goodfellow, Y. Bengio und A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. besucht am 27. Okt. 2026. Adresse: https://www.deeplearningbook.org/

[3] M. L. Minsky und S. A. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969.

[4] Kolop1012, *LikunnN_Feed-Forward-Netz_A1*, https://github.com/Kolop1012/LikunnN_Feed-Forward-Netz_A1, GitHub Repository, 2023.

[5] D. Vieten, *Lernen in kNN: Feed-Forward-Netze und Backpropagation*, Vorlesungsfolien, Projektseminar Lernen in natürlichen und künstlichen neuronalen Netzen, Hochschule Bonn-Rhein-Sieg, SW03, 2026.

[6] D. E. Rumelhart, G. E. Hinton und R. J. Williams, „Learning representations by back-propagating errors,“ *Nature*, Jg. 323, Nr. 6088, S. 533–536, 1986. DOI: 10.1038/323533a0

[7] M. A. Nielsen, *Neural Networks and Deep Learning*. Determination Press, 2015. besucht am 27. Okt. 2026. Adresse: https://neuralnetworksanddeeplearning.com/chap2.html

[8] D. Vieten, *BIAS, Projektbericht und Korrektheitsüberlegungen*, Vorlesungsfolien, Projektseminar Lernen in natürlichen und künstlichen neuronalen Netzen, Hochschule Bonn-Rhein-Sieg, SW02, 2026.

[9] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.

[10] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, 2. Aufl. Cham: Springer, 2023. DOI: 10.1007/978-3-031-29642-0